

Keakuratan Analisis

1. Apa perbedaan antara ADT (Abstract Data Type) dan struktur data, dan mengapa pemisahan konsep ini penting dalam rekayasa perangkat lunak?

Perbedaannya untuk abstract data type adalah konsep logis apa yang dilakukan data seperti melakukan operasi dalam ADT Stack push, pop, peek sedangkan untuk struktur data adalah implementasi data untuk menjelaskan bagaimana data akan di simpan, di atur dan juga di manipulasi di memory. Konsep ini penting dalam rekayasa perangkat lunak untuk mengefisiensi waktu kerja, reusability, keamanan data, modular dan memudahkan programmer untuk bekerja.

2. Bagaimana pemilihan struktur data yang tepat dapat memengaruhi efisiensi algoritma? Apa yang terjadi jika Anda memilih struktur data yang tidak sesuai untuk suatu algoritma? Berikan contoh konkret.

Struktur data yang tepat dapat membuat waktu untuk menemukan datanya jadi lebih cepat, penggunaan memory yang lebih efisien, lebih mudah juga dalam memodifikasi serta untuk skalabilitasnya. Yang terjadi apabila salah dalam memilih struktur data adalah kecepatan menjadi lambat, memori yang boros, kode menjadi lebih rumit dan tidak scalable.

Contohnya seperti untuk membawa satu penumpang yang bisa menggunakan motor tetapi lebih memilih menggunakan mobil dengan CC yang besar sehingga boros di bensin, susah masuk yang kecil dan lambat untuk mencapai tujuan

3. Mengapa analisis kompleksitas waktu yang dipelajari minggu ini menggunakan total frekuensi hitungan $T(n)$, bukan dalam satuan detik atau menit?

Karena apabila menggunakan satuan detik dan menit sangat bergantung kepada OS, hardware dan bahasa pemrograman yang digunakan sehingga apabila beda maka akan beda juga hasilnya

4. Ketika Anda menulis sebuah algoritma, apa saja tanda-tanda atau pola yang menunjukkan bahwa algoritma tersebut memiliki kompleksitas waktu $O(n^2)$?

Nested loop algoritma adalah tanda bahwa algoritma tersebut memiliki kompleksitas waktu $O(n^2)$

5. Ketika Anda terlibat dalam sebuah proyek nyata, mana yang akan Anda pilih: efisiensi waktu (time-efficient) atau efisiensi ruang/memori (space-efficient)? Jelaskan alasan dan pertimbangan Anda.

Saya akan memilih efisiensi waktu karena di saat ini banyak berkembang web app dan mobile app dimana waktu eksekusi dan kenyamanan pengguna wajib di prioritaskan ditambah lagi banyaknya kompetitor dan untuk saat ini waktu lebih mahal untuk merebut kenyamanan konsumen serta memory saat ini harganya masih cukup terjangkau dengan juga adanya teknologi virtual ram.

Kesesuaian Prinsip, Teori & Metode

6. Untuk algoritma-algoritma berikut, hitung $T(n)$ dan Big-O dari kompleksitas waktunya.

a. Fungsi isPalindrome(s):

Inisialisasi: 2 operasi ($i \leftarrow 0, j \leftarrow \text{length}(s) - 1, \text{return true}$)

Loop: 4 Operasi (while $i < j$, if $s[i] \neq s[j]$, $i \leftarrow i + 1, j \leftarrow j - 1$)

$$T(n) = 2 + (n/2 \cdot 4)$$

$$T(n) = 2 + 2n$$

Kompleksitasnya adalah $O(n)$ atau linear time

b. Prosedur iSort(A, n) — Insertion Sort:

Inisialisasi: 1 operasi ($i \leftarrow 1$)

Outer Loop: 4 operasi (for, $\text{key} \leftarrow A[i], j \leftarrow i - 1, A[j + 1]$)

Inner Loop: 4 operasi (while $j \geq 0$ and $A[j] > \text{key}$ do, $A[j + 1] \leftarrow A[j], j \leftarrow j - 1$)

$$T(n) = 1 + (4n - 4) + (2n^2 - 2n)$$

$$T(n) = 2n^2 + 2n - 3$$

Kompleksitasnya adalah $O(n^2)$ karena adanya nested loop

7. Untuk algoritma-algoritma berikut, hitung estimasi penggunaan memori $S(n)$ dan Big-O dari kompleksitas ruangnya (Space Complexity).

a. Fungsi sumArray(A, n):

input: array n, variabel 1

variable: total 1, l (loop) 1

$$S(n) = n + 3$$

Space complexity: $O(1)$

b. Fungsi fibonacci(n) — Rekursif:

input: n-1

function: 1

return: 2 kali

$$S(n) = cn + 2c + 1$$

Space complexity: $O(n)$

8. a. $f(n) = 5n^2 + 10n + 20$; $g(n) = n^2$

Tunjukkan konstanta $c > 0$ dan $n_0 > 0$ yang memenuhi definisi formal Big-O dan Big- Θ , beserta pembuktian langkah demi langkah.

- Pembuktian big O

$$5n^2 + 10n + 20 \leq c * n^2$$

$$5n^2 + 10n + 20 \leq 5n^2 + 10n^2 + 20n^2 = 35n^2$$

$$c=35, n_0=1$$

$$f(n) \leq 35n^2$$

$$f(n) = O(n^2)$$

- Pembuktian Big- Θ

$$c_1 n^2 \leq 5n^2 + 10n + 20 \leq c_2 n^2$$

Batas bawah

$$5n^2 + 10n + 20 \geq 5n^2$$

$$5n^2 \leq f(n)$$

$$c_1=5$$

Batas atas

$$f(n) \leq 35n^2$$

$$c^2 = 35$$

$$n_0=1$$

$$5n^2 \leq 5n^2 + 10n + 20 \leq 35n^2$$

$$f(n) = \Theta(n^2)$$

b. $f(n) = 2n^3 + 50n$; $g(n) = n^2$ Tunjukkan apakah $f(n)$ tumbuh lebih cepat, sama, atau lebih lambat dari $g(n)$. Sertakan pembuktian lengkap untuk mendukung kesimpulan Anda.

Suku dengan pangkat terbesar adalah:

$$f(n) = 2n^3$$

$$g(n) = n^2$$

n	n^2	$2n^3 + 50n$
10	100	2.500
100	10.000	2.005.000
1000	1.000.000	2.000.050.000

Terlihat bahwa:

$$f(n)$$

naiknya jauh lebih cepat.